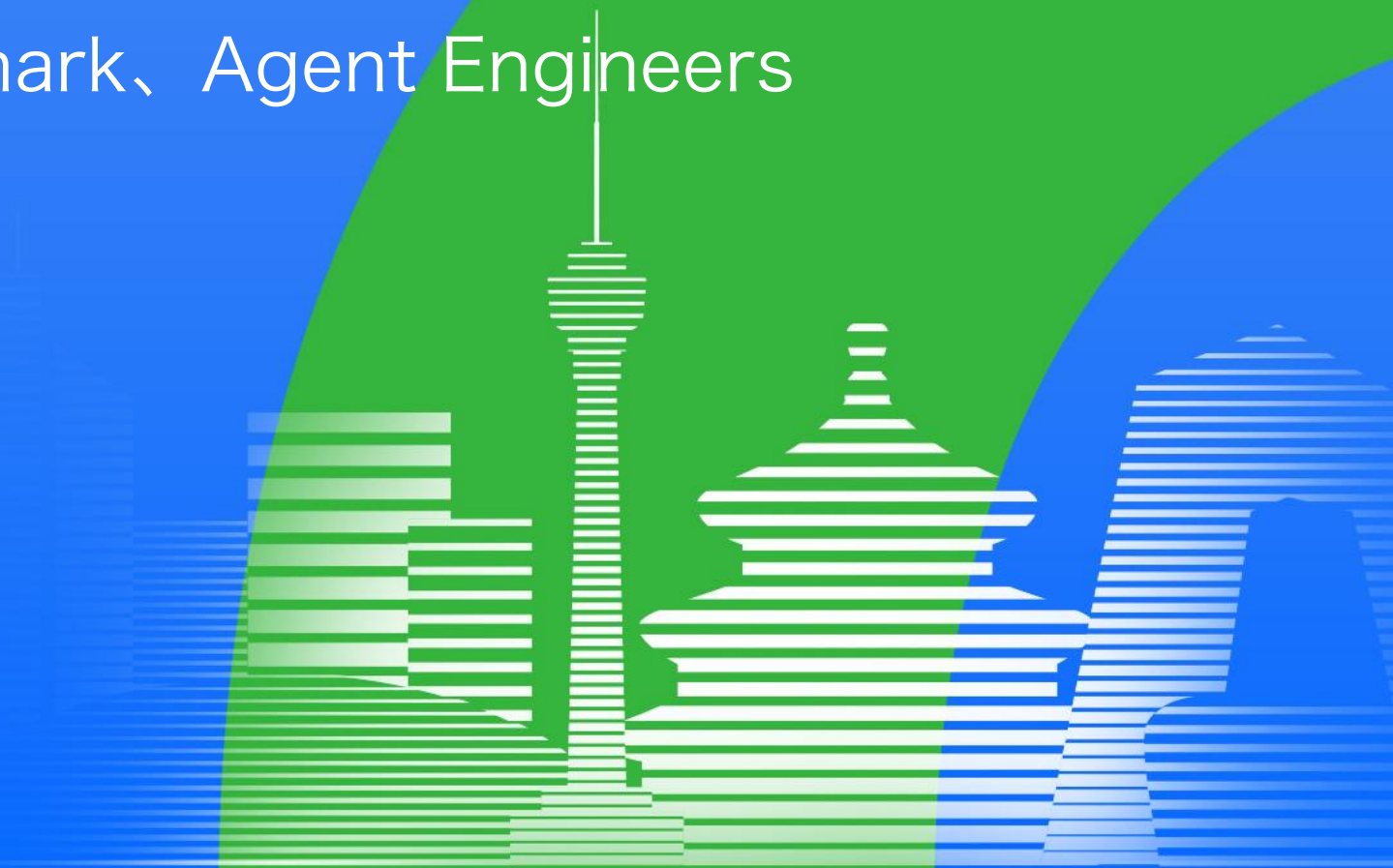


构建 Coding Agent 的飞轮： Feedback Loop、Benchmark、Agent Engineers

牛万鹏

百度 Comate 研发经理



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

- 01 Coding Agent在百度落地的效果
- 02 Agent Loop的基本框架和问题
- 03 Feedback Loop: 让 Agent 的行为可观测
- 04 Benchmark: 挖掘评测集和发现异常值
- 05 Agent Engineers: 把人放到Loop里

01

Coding Agent在百度落地的效果

Coding Agent深入人心，从研发扩散到全员

增长趋势图

单图突出快速增长，标记 2 月末跳升拐点与 3 月持续创新高区间。

18 个周度观测点



Comate全员落地，人人使用

人均Query次数增长5倍

相对于人均Tokens统计，我们更专注与人均Query次数统计，有效、合理的Query更能代表用户对Coding Agent的使用情况。

Comate IDE成为主要入口

Comate IDE已经成为百度内Coding Agent的基础设施，唤起时长占有所有IDE的60%以上，JetBrains、VSCode等传统IDE逐渐退潮。

全员Coding

不仅仅是研发，产品经理、项目经理、交互&视觉设计师、测试工程师、售前工程师、销售等全员Coding。

Vibe Coding不再是纸上谈兵，古法编程渐行渐远

类型一：解决之前大量可以用研发解决，但不会被纳入研发排期的工作

Coding Agent会成为类似俄乌战争中无人机的作用，深刻的改变战场形态。

- 「发现战情」：战线十公里外发现2个敌人
- 「传统战场」：这两个敌人十分安全
 - 决策链路：远程火力需要上报到师级首长决策。
 - 时效性：等远程炮兵做好准备敌人可能已经跑了。
 - 风险：炮兵阵地暴露可能遭致敌人反击
- 「俄乌战场」：一线战士直接操作无人机发动攻击

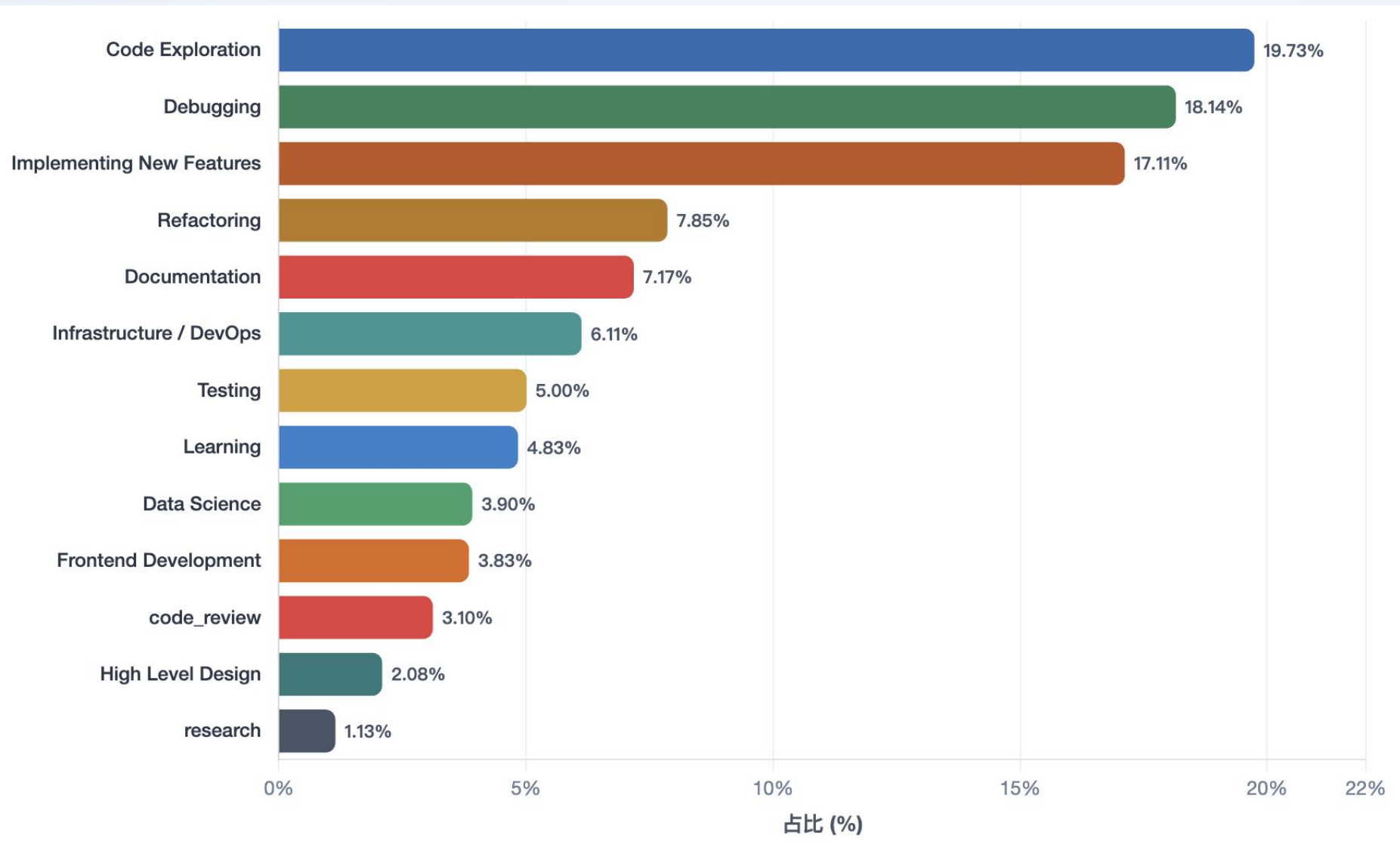
类型二：需求开发、问题排查、自动化测试等真实软件研发活动

Coding Agent加速推动开发者转型，深刻影响研发团队的协作模式。

- 「针对开发者个人」：从 执行者 到 问题提出者，制约效率的不是Tokens消耗速度，而是提出好问题的能力
- 「针对研发团队」：从 大Feature要拆小Task分给多人执行 转变为 直接交给一人+Agent 完成，效率跟高



使用Coding Agent完成的任务类别统计



Coding Agent覆盖所有环节

- 代码探索、问题排查和新功能实现是Coding Agent的主要场景
- 代码重构、文档撰写以及DevOps相关代码生成也是高频场景

02

Agent Loop的基本框架和问题

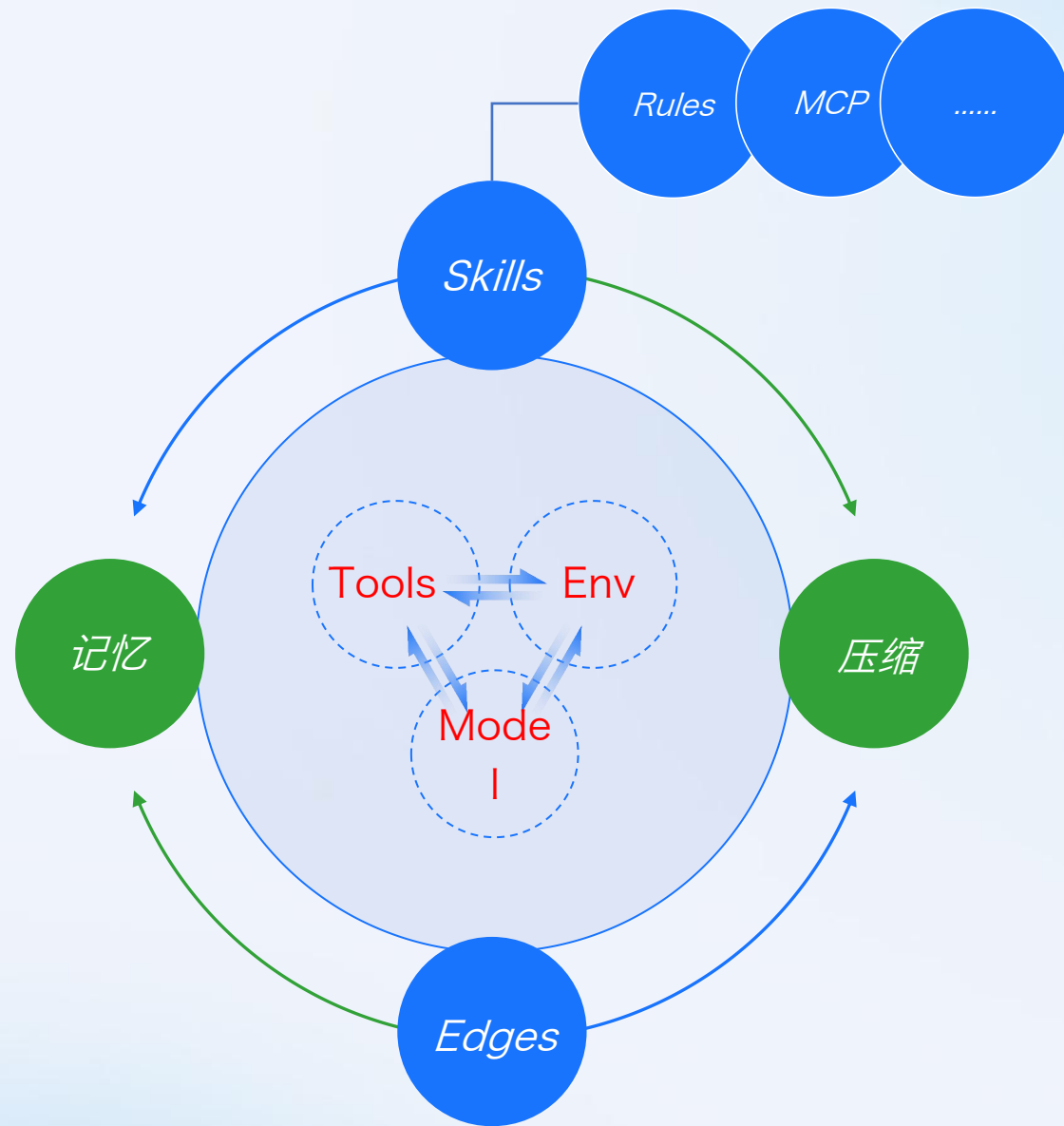
Agent Loop的基本框架

核心Loop：驱动Agent运转，感知外部环境变化

包含模型、工具以及Agent所运行的环境。很多Agent框架忽略模型本身「特性」的变化，尝试打造统一的框架适配所有模型，Comate的实践让我们清醒认识到这是错误的。因此，Agent框架必须能够发现模型的「变化」。

外层Loop：为Agent提供扩展性、鲁棒性

通过记忆、Skills等Context来为Agent提供更多接触环境、感知环境的手段，从而大幅延长Agent的「触角」。通过「压缩」和「Edges（边界情况处理）」来让Agent具备自愈能力。





打造一个通用Agent产品时，构建Agent Loop面对的核心问题

对模型解题能力的持续观测，动态调整Agent框架。

1. 随着模型的「解题」能力越来越强，构建Agent核心原则从「流程定义」变为「状态感知」，不再需要教模型怎么做，只需要告诉模型现在发生了什么
2. 针对Claude 4.6系列、GPT-5.3/5.4系列模型对与编码的细节处理已经足够优秀，完全不需要Spec Driven Development、Superpowers等开发模式/脚手架，反而拖累模型效果，浪费上下文

构建有效的评测级，重点不再是解题分数而是发现异常值。

1. 单纯做面向SWE-Bench等通用方法的评测无法真正体现Agent的效果，评测集和对应的评判标准体现了Agent的「调性」
2. 评测单纯看分数是不够的，需要发现异常值，通过异常值发现模型的偏好

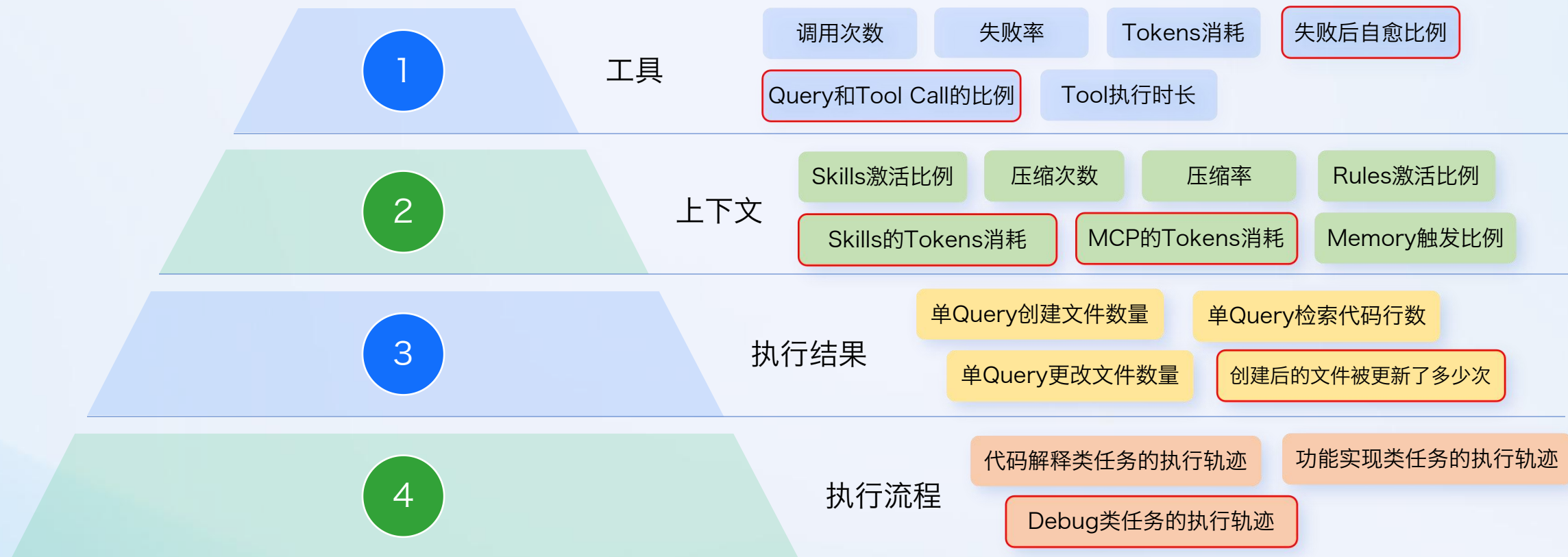
「人」也是Agent的Tool、Context，需要打破角色分工。

1. 传统软件研发协作流程将开发者分成前端、后端、测试等各个角色，这种协作适用于「稳定态」的软件，而基于模型构建的Agent应用是「混沌态」，无法有效适用

03

Feedback Loop: 让 Agent 的行为可观测

构建线上Agent执行数据的观测体系

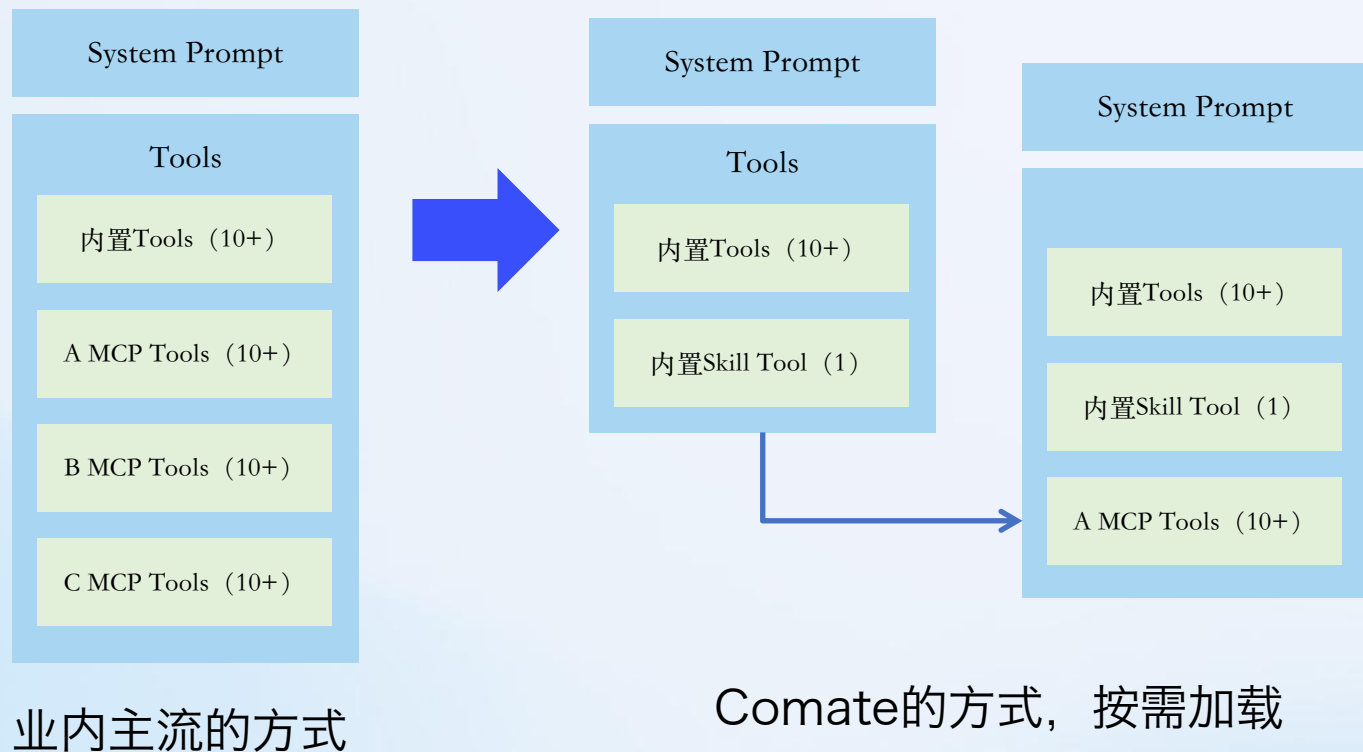




实践：MCP和Skills的Tokens消耗对比强烈，推动我们快速优化

■ 使用Skills的方式实现MCP动态加载，节省98%的Tokens消耗

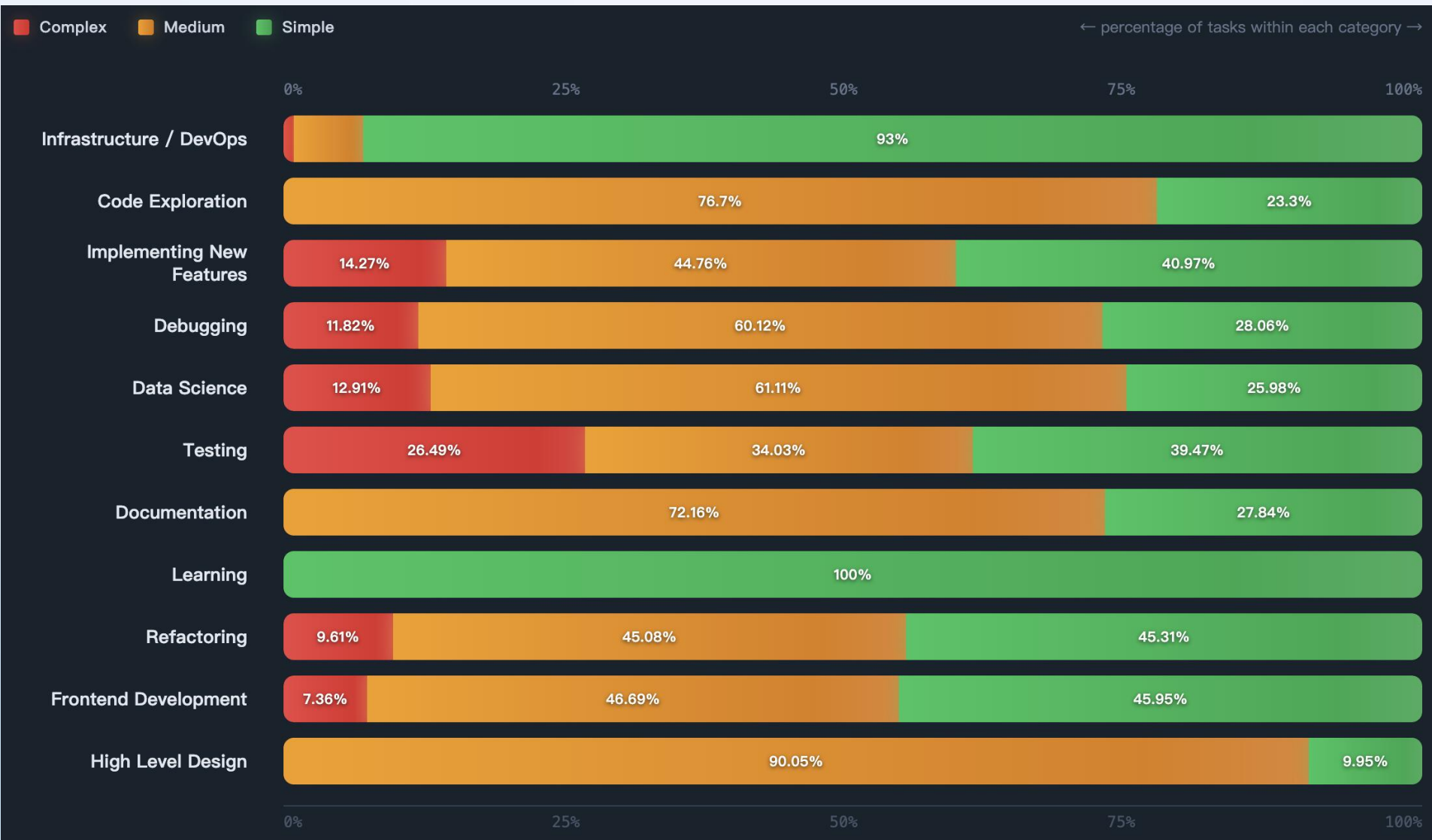
- ✓ Comate自动为每个MCP Server生成Skill风格的描述，并作为一个虚拟的Skill加载
- ✓ 模型根据Query自动判断是否需要加载整个MCP Server的Tools



```
1 用户在 Comate MCP 配置界面安装 MCP 服务器
2      |
3      ▼
4  Comate 自动为每个 MCP 服务器生成对应的 Skill 配置
5  (名称 + 高级描述, 记录服务器能力概要)
6      |
7      ▼
8  每次请求时, skill 工具 description 中嵌入所有 MCP Skill 的轻量列表
9  (模型可见: 服务器名称 + 能力概要)
10     |
11     ▼
12  模型根据任务判断: 是否需要使用某个 MCP 服务器?
13     |                               |
14     否                               是
15     |                               |
16     (不加载)                       调用 skill(name="<server_name>")
17     |                               |
18     |                               ▼
19     |                       Comate 实时拉取该 MCP 服务器最新工具列表
20     |                       返回所有工具的完整 Schema 定义
21     |                               |
22     |                               ▼
23     |                       模型阅读 Schema, 选择合适工具
24     |                       调用 use_mcp_tool(server_name, tool_name, arguments)
25     |                               |
26     |                               ▼
27     |                       Comate 执行 MCP 工具调用
28     |                       返回结果给模型
```




用Agent分析Agent执行数据，解构任务复杂度



将任务分成复杂、中等、简单

- 通过任务类别的不同观测执行时长、Tools次数、压缩次数等指标
- 不同任务类别在横向、纵向比对的差异也让我们观测Agent更善于做什么

实践： Bash是GPT系列最喜欢的工具， 定向调整



- 随着轮次的增多我们发现GPT系列模型越来越喜欢调用Bash工具
- 通过Bash做检索，而不是Grep/Glob
- 通过Bash做编辑，而不是Edit
- 通过Bash做读取，而不是Read
-

这并不代表模型的指令遵循有问题，而是体现了它的偏好

我们应该提高Bash的权重，而不是抑制这种行为

04

Benchmark: 挖掘评测集和发现异常值

从业务本身挖掘评测集，善用Git Blame

为Agent提供作为评测集的代码库

- 必须是一个Git仓库，且有良好的Commit Message
- 告诉Agent希望提取什么类型的Case作为评测

Agent通过分析Git Logs提取信息

- Agent开始逐个分析Commit判断是否适合作为评测集
- 如果适合则抽象给出评测的Query、验证方法

启动另一个Agent进行验证

- 检查无误则启动另一个独立Agent进行验证
- 验证结果经过Agent和人工检验

人工检查评测集的合理性并修正

- 检验Agent提取Case的合理性
- 请注意，一条评测Case一般要跨多个Commit，需告知Agent这样的情况存在





评测结果的度量，分析流程和异常值 —— 四象限分析法

$$\text{Overall Score} = 0.6 \times \text{Outcome Score} + 0.4 \times \text{Execution Score}$$

Outcome Score (结果质量 · 权重 60%)

- *Test Pass* (实际测试通过, 0.5)
- *Functional Correctness* (功能正确性, 0.2)
- *Completeness* (完整性, 0.1)
- *Patch Quality* (代码质量, 0.1)
- *Regression Risk* (回归风险, 0.1)

Execution Score (执行效率 · 权重 40%)

- *Problem Understanding* (理解能力, 0.25)
- *Agent Execution Quality* (执行质量, 0.75)
 - *Investigation*
 - *Tool Usage*
 - *Control Flow*
 - *Task Management*
 - *Validation*
 - *Craftsmanship*

OUTCOME 轴定义

Test 通过 $\text{outcome} = 0.5 + \text{llm_score}/2$, 范围 [0.5, 1.0]

Test 不通过 $\text{outcome} = 0 + \text{llm_score}/2$, 范围 [0, 0.5]

```
llm_score = (  
    functional_correctness +  
    completeness +  
    patch_quality +  
    regression_risk  
) / 4
```

EXECUTION 轴定义

核心指标 $\text{execution} = \text{agent_execution_quality}$ (LLM评分)

高 Execution $\text{execution_quality} \geq 0.7$ (执行规范、高效)

低 Execution $\text{execution_quality} < 0.7$ (存在循环/混乱)

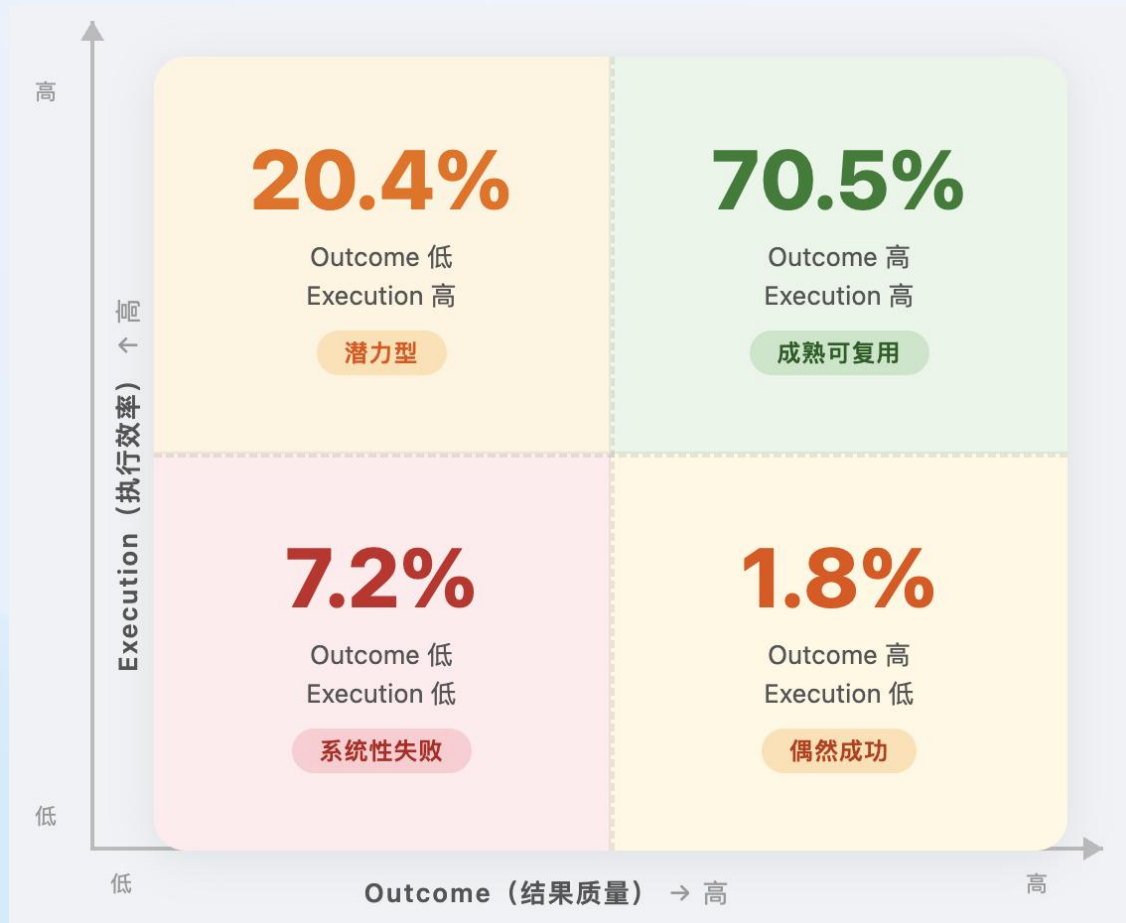
agent_execution_quality 含:

- Investigation & Localization
- Tool Usage / Control Flow
- Task Management / Validation
- Execution Craftsmanship

评测结果的度量，分析流程和异常值

Outcome / Execution 四象限分析

X 轴: Outcome (结果质量) | Y 轴: Execution (执行效率)



HH

Outcome 高 / Execution 高 — 成熟可复用 **70.5%**

测试通过 + 高质量执行，这是 Agent 的"标准成功态"。代表任务: **sympy__sympy-21847** (96.8 分)，仅用 3 次工具调用完成精准定位与修复。

HL

Outcome 高 / Execution 低 — 偶然成功 **1.8%**

测试通过但执行低效，说明高质量执行是成功的必要条件，低执行质量极少能带来好结果。代表任务: **astropy__astropy-14598** (75.9 分)。

LH

Outcome 低 / Execution 高 — 潜力型 **20.4%**

执行规范但测试未通过，说明存在定位准确但修复逻辑有误的系统性问题。这类任务通过改进 Prompt 策略具有较大提升空间。

LL

Outcome 低 / Execution 低 — 系统性失败 **7.2%**

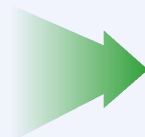
测试未通过 + 执行混乱，通常涉及需求误解或技术难度过高。代表任务: **django__django-13513** (12.0 分)，无有效任务管理与验证。



实践：Agent发现8%的工具无效调用，推动我们调整压缩算法

在通过分析Agent执行轨迹的时候发现有很多无效的工具调用

无效工具调用是指在Explore过程中的冗余，在当时的Query轮次下有用，但后续的轮次没用。这些无用的工具轮次会占用大量的上下文。



疑问

- 1、主Agent可以自我意识到哪些Tool对儿已经失效吗？
- 2、如果能够意识到，这能否用于压缩？

Investigation & Localization

定位与调查

0.840 P50: 0.85

★ 最强能力，~85% 任务得分 ≥ 0.8 ，Agent 快速定位问题根源

Tool Usage

工具使用

0.785 P50: 0.80

工具成功率 92%，但存在 redundant_verification (32次) 等低效模式

Validation & Feedback

验证与反馈

0.728 P50: 0.85

分化严重，70% 得分高但 30% 验证不足，validation_gap 导致通过率仅 12%

Execution Craftsmanship

执行工艺

0.696 P50: 0.70

代码质量不稳定，over_engineering 是主要问题 (18次出现)

Control Flow

控制流

0.682 P50: 0.70

⚠ 最弱子维度，30% 任务得分 < 0.6 ，execution_loop 出现 26 次

Task Management

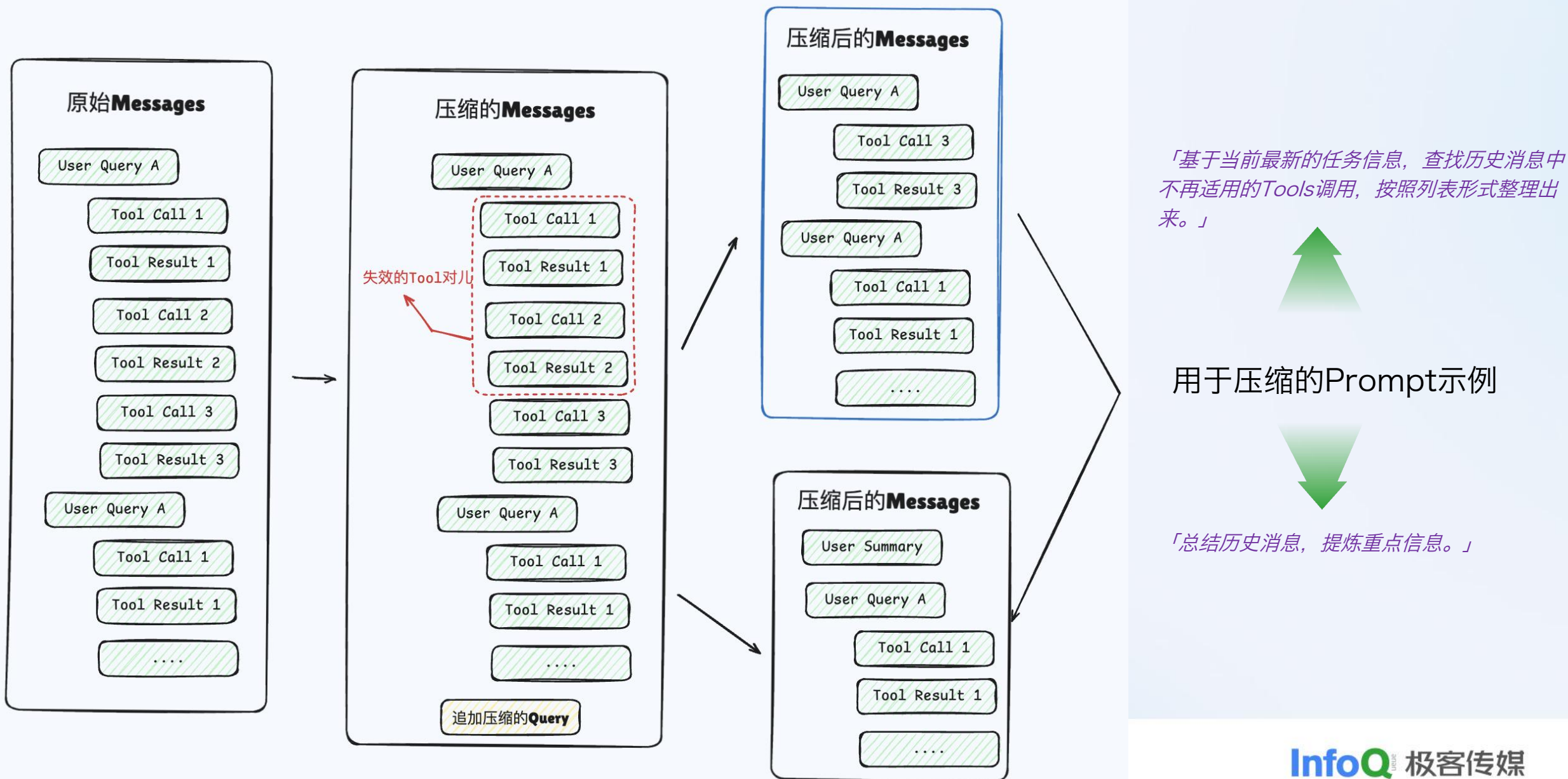
任务管理

0.605 P50: 0.70

⚠ 第二弱维度，40% 任务 < 0.6 ，缺乏显式 Task/TODO 工具使用



实践：Agent发现8%的工具无效调用，推动我们调整压缩算法



实践：从评测中发现Tools的执行网络

评测中的异常值



两个不同Tool经常成对出现

当Edit失败后，通常会Read。当Read失败后，通过会List或Bash。

从思考内容中发现模型的计划

在思考内容中可以观测到模型推理过程中会有一些面向工具的try catch逻辑，即“某个工具调用失败了我要做什么”。

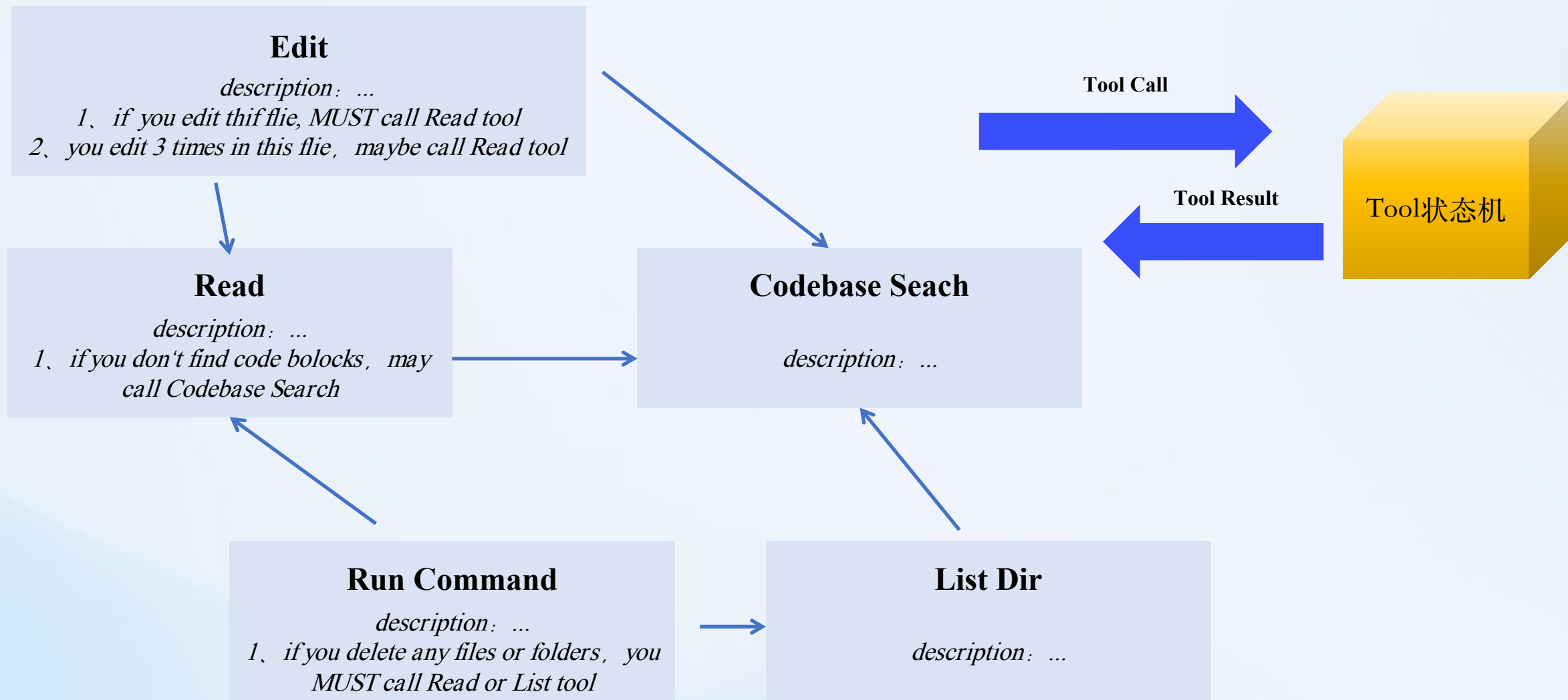
Fix类的问题频繁出现临时文件

排查问题时经常通过Bash创建临时文件存储排查日志，分析完毕后调用Delete工具删除。

Tools的执行网络

- ✓ 不在任何System Prompt和Tool中教模型「你应该这么干（How）」，而是只告知模型「当前的运行状态和什么是好的（What）」，即Agent框架的目的是构建一个模型能运作的环境，而不是模型必须遵守的执行路径
- ✓ 随着模型的「解题」能力越来越强，构建Agent核心原则从「流程定义」变为「状态感知」，不再需要教模型怎么做，只需要告诉模型现在发生了什么

实践：从评测中发现Tools的执行网络



05

Agent Engineers: 把人放到Loop里

把人放到Loop里的两层含义

1

全员转型 —— 每个人都成为Agent Engineer

- 不只是开发者在用Agent, PM、UE、测试、售前都在用IDE写代码 —— 角色边界正在消失
- 转型的核心不是"学会写Prompt", 而是从"执行者"转变为"问题提出者", 能力瓶颈从编码速度变成了提出好问题的能力
- 协作模式从"大Feature拆小Task分给多人"变为"一人 + Agent一竿子捅到底"

2

把人看作Agent的工具和Context

- 理解中人是Agent的使用者、监督者。但在实践中, 人本身就是Agent Loop的一部分 —— 人提供的判断、审美、业务知识是Agent无法自己生成的Context
- 当Agent遇到需要产品决策、视觉判断、业务逻辑确认的节点时, "调用人"和"调用Tool"没有本质区别

● 全员转型推动新的Agent能力产生

Comate在做全员向Agent工程师的转型

打破角色边界，不区分任何端

所有角色都是「一竿子捅到底」，协同Agent开发所有端。但最终有特定角色做代码审查。

反转协作链条，先开发后定义

大家针对某个「点子」讨论后很感兴趣，觉得有价值，则马上开发，不需要产品经理、视觉设计师等做任何设计。开发完毕后做showcase然后再做验证。



催生了新诉求：Agent全流程验证

由于同时开发前后端，甚至同时开发更多的工程，这让本地开发调试带来很大的困难，这催生了我们做异步化，重点建设云端沙盒。沙盒不是一个远程跑代码的容器，而是：一种可授权、可观测、可回放、可验证、可交付的 Agent 任务操作空间

催生了新诉求：PM、UE要原子化

由于很多Feature都是开发先实现，然后在交给PM和UE做审查优化，为了尽可能降低返工成本需要PM、UE要将产品的各类素材原子化，作为Skills内置到项目中。

Agent交付的内容不仅仅是代码

可执行的工程成果

交付给人的不是"一段需要你自己跑起来看看的代码", 而是一个已经跑起来的、可直接体验的工程产物。PR、构建包、可访问的预览环境——
Agent把"写完"和"能用"之间的距离压缩为零。

可验证的质量证据

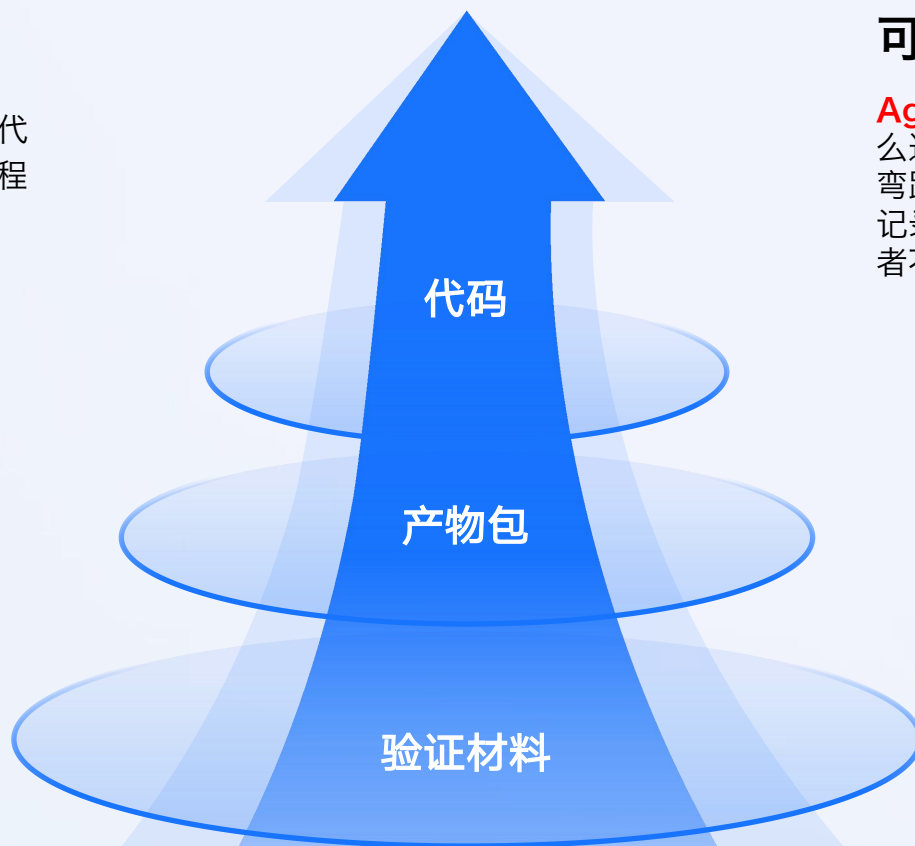
Agent交付的是自证清白的证据包: 自动截图证明页面渲染正确, 录屏展示完整操作流程, 测试报告覆盖边界条件。交付从"请你相信我"变成了"请你检验我"——Agent主动承担了举证责任。

可追溯的决策过程

Agent的执行轨迹本身就是一种交付物。它为什么选择这个方案而不是那个? 排查问题时走了哪些弯路、排除了哪些假设? 这些思考链路和工具调用记录构成了一个完整的决策档案, 它让后续维护者不用重新走一遍探索的路。

可复用的知识沉淀

Agent在完成任务的过程中会接触大量的代码库结构、业务逻辑、技术约束。这些信息经过Agent的整理, 可以沉淀为变更说明、接口文档、架构决策记录(ADR), 甚至反哺为项目的Skills和Rules。



「沙盒」不是一个远程跑代码的容器, 而是一种可授权、可观测、可回放、可验证、可交付的Agent任务操作空间。它让Agent的输出从"文本"升维到"可运行的、有据可查的完整工程成果"。

不是未来，就是现在，我们已经活在Agent的世界

语言已经被改变

- 开周会看一周纪要，我们说"大家互换一下Context"
- 解决困难问题，我们说"这太废Tokens了"
- 评审方案的时候，我们说"你这个Prompt写得不好"

工作方式已经被重构

- 60%的IDE唤起时长已经是AI IDE
- 人均Query次数增长了5倍
- 不是研发在用，是全员在用先开发后定义，先Showcase再验证，传统的瀑布式流程正在被反转



活在Agent的世界

组织协作已经被重塑

- 大Feature不再拆小Task分给多人，而是一人+Agent一竿子捅到底
- PM、UE要将素材原子化，作为Skills内置到项目中，配合Agent随时调用
- 人不再只是Agent的使用者，而是Agent Loop中的Tool和Context——角色边界消失

效率的放大器，让强者恒强

- 善于提出好问题的人，会被Agent成倍放大
- 善于构建Feedback Loop的团队，会以指数速度迭代Agent能力
- 真正的竞争优势不是"用不用Agent"，而是"你的飞轮转起来了没有"

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

